MINI PROJECT REPORT

Student Records Search System

A Python-based project using Linear Search and Binary Search

Submitted by: Patel Dhairya R

Enrollment number: 12402080601028

Department: IT-A [1B8]

Institution: A D Patel Institute of Technology

Submitted to: Minakshi patil

Academic Year: 2026-27

Acknowledgement

I would like to express my sincere gratitude to my teacher/guide for their guidance and support during the preparation of this project. I also acknowledge the learning resources that helped me understand Python programming, data structures, and searching algorithms. This report describes the design and working of the Student Records Search System.

Replace or edit this section to reflect the actual people and resources involved.

Abstract

The Student Records Search System is a menu-driven Python program that stores and manages basic student details during a program session. Each record contains a roll number, student name, and marks. The system allows users to add records, display all records, and search for a student by roll number using either Linear Search or Binary Search. It also checks for duplicate roll numbers when adding records and reports PASS or FAIL using 35 marks as the passing threshold. The project demonstrates Python functions, lists, dictionaries, loops, conditional statements, sorting, and exception handling.

Table of Contents

1. Introduction

2. Problem Statement

3. Objectives

4. Scope of the Project

5. Tools and Technologies

6. System Requirements

7. System Design and Workflow

8. Data Structure

9. Implementation and Algorithms

10. Sample Input and Output

11. Testing

12. Limitations

13. Future Enhancements

14. Conclusion

15. References

1. Introduction

Student information is often needed for academic administration, verification, and progress checking. Searching records manually can take time, especially when information is not organized consistently. The Student Records Search System provides a simple console-based way to store a small set of student details and retrieve a record by roll number.

The application is written in Python. Its main menu lets the user enter records, display all saved records, search for a student, or exit the program. The project is intended as a learning example and does not use a permanent database or file storage.

2. Problem Statement

Manual searching and maintaining student information can be inconvenient and may lead to mistakes. A small computerized program can organize the information in a consistent structure and demonstrate a systematic search process. This project addresses the basic need to add student details, list saved records, and find a record by its roll number.

3. Objectives

- Allow the user to enter a student's roll number, name, and marks.
- Prevent a new record from being added when its roll number already exists.
- Display all records currently stored in the program.
- Search for a student using Linear Search or Binary Search.
- Display the matching student's details and PASS/FAIL status.
- Demonstrate Python functions, lists, dictionaries, loops, conditions, and exception handling.

4. Scope of the Project

The current version is suitable for learning, small demonstrations, and understanding search algorithms. It manages records only while the program is running. It does not provide login accounts, multiple-user access, a graphical interface, permanent storage, or advanced student-management features.

5. Tools and Technologies

- Programming language: Python 3.
- Data structures: List and dictionary.
- Control structures: while loop, for loop, if/elif/else statements.
- Search techniques: Linear Search and Binary Search.
- Error handling: try/except for ValueError around menu operations.
- Interface: Text-based console menu.

6. System Requirements

Hardware

- A computer or laptop capable of running Python.
- Basic keyboard and display.

Software

- Python 3 installed on the system.
- A Python editor or IDE, such as IDLE, VS Code, or PyCharm.
- A terminal or command prompt to run the program.

7. System Design and Workflow

The program starts by creating an empty list named students. It repeatedly displays the main menu until the user chooses Exit.

1. Enter Student Records: ask how many records to add, then collect roll number, name, and marks for each entry.
2. Display All Records: check whether any records exist and print each saved record.
3. Search Student Record: request a roll number, then ask the user to select Linear Search or Binary Search.
4. Show Search Result: if a match is found, display the roll number, name, marks, and PASS/FAIL status; otherwise show that the record was not found.
5. Exit: stop the menu loop and end the program.

A duplicate roll number causes the current entry to be skipped and the loop continues to the next student. The program displays a general success message after the entry loop even if an entry was skipped.

8. Data Structure

The program uses a list called students. Each item in the list is a dictionary containing three keys:

Key	Data type	Purpose
roll	Integer	Unique student roll number
name	String	Student's name
marks	Float	Student's marks

Illustrative example of one record: {'roll': 101, 'name': 'Aarav', 'marks': 78.5}. These values are examples only; the program asks the user to enter actual data.

9. Implementation and Algorithms

9.1 add_students()

This function asks how many students the user wants to enter. For each student, it reads the roll number and checks whether that roll number already exists in the list. If it is a duplicate, the entry is skipped. Otherwise, the function collects the student's name and marks and appends a dictionary to students.

9.2 display_students()

This function checks whether the list is empty. If it is empty, it prints a message. Otherwise, it prints a heading and loops through the list to display the roll number, name, and marks of each student.

9.3 Linear Search

Linear Search checks records one by one from the beginning of the list. When a record's roll number matches the requested number, the function returns that student dictionary. If no match is found after checking all records, it returns None.

Worst-case time complexity: O(n), where n is the number of student records.

9.4 Binary Search

Binary Search first creates a sorted copy of the student list, ordered by roll number. It then compares the target roll number with the middle record. If the target is smaller, it searches the left half; if larger, it searches the right half. The search continues until a match is found or the search range becomes empty.

Binary Search requires sorted data. In this implementation, sorting is performed every time Binary Search is selected. Sorting takes O(n log n), while the binary-search step takes O(log n). Therefore, the complete operation is dominated by the sorting step for this implementation.

9.5 search_student() and Result Checking

The search_student() function checks that records exist, asks for a roll number, and prompts the user to choose a search method. It displays the matching record when found. If marks are greater than or equal to 35, the result is PASS; otherwise, the result is FAIL. If no record matches, it prints 'Student record not found!'.

9.6 Main Program and Exception Handling

The main while loop presents four options and calls the appropriate function. Invalid menu choices are rejected. A ValueError handler catches many invalid numeric inputs, such as text entered where an integer or float is expected. Since the handler surrounds menu operations, invalid numeric input can return the user to the menu rather than prompting for that particular field again.

10. Sample Input and Output

The following is an illustrative example. The names, roll numbers, and marks are fictional.

Sample record list

Roll Number	Student Name	Marks
101	Aarav	78.5
102	Diya	29.0
103	Kabir	66.0

Example search result

```
Roll Number: 101
Student Name: Aarav
Marks: 78.5
Result: PASS
```

If a user searches for a roll number that is not in the list, the program displays 'Student record not found!'.

11. Testing

The following test cases can be used to check the program. Actual results should be confirmed by running the code in Python.

Test case	Input / action	Expected behavior
Add valid student	New roll, name, and numeric marks	Record is added
Duplicate roll	Enter a roll number already in students	Duplicate warning; entry skipped
Display empty list	Choose display before adding records	No-records message
Search existing roll	Choose either search method for a stored roll	Matching details are displayed
Search missing roll	Enter an unknown roll number	Record-not-found message
Pass threshold	Marks = 35	PASS
Below threshold	Marks = 34.9	FAIL
Invalid numeric input	Enter text for roll or marks	ValueError is caught by outer handler

12. Limitations

- Records are stored only in memory and are lost when the program exits.
- The duplicate-roll check skips an entry rather than asking the user to enter a different roll number.
- The program does not validate whether marks fall within a reasonable range, such as 0 to 100.

- Invalid numeric input may return the user to the main menu instead of re-prompting for the field.
- Binary Search sorts the list for every search, adding overhead.
- There is no authentication, database, file export, or audit history.

13. Future Enhancements

- Save and load records from a file or database such as SQLite, MySQL, or PostgreSQL.
- Add update, delete, and confirmation features.
- Re-prompt for duplicate roll numbers and invalid values.
- Validate marks within the institution's allowed range.
- Keep records sorted or use a dictionary indexed by roll number for efficient lookup.
- Add a graphical user interface and role-based access if used in a real institution.
- Add automated unit tests and improve formatted output.

14. Conclusion

The Student Records Search System demonstrates a basic approach to managing student records with Python. It combines list and dictionary data structures with functions, loops, conditional logic, duplicate checking, and exception handling. By implementing Linear Search and Binary Search, the project also illustrates how different algorithms locate a record. Although the current program is intended for demonstration and does not save data permanently, it provides a useful foundation for a more complete student management application.

15. References

- Python Software Foundation, Python 3 Documentation: https://docs.python.org/3/
- Python Software Foundation, Built-in Types (lists and dictionaries): https://docs.python.org/3/library/stdtypes.html
- Python Software Foundation, Errors and Exceptions: https://docs.python.org/3/tutorial/errors.html